

PARTIE 1 : Fondamentaux des SGBD

Chapitre 1: Introduction aux SGBD

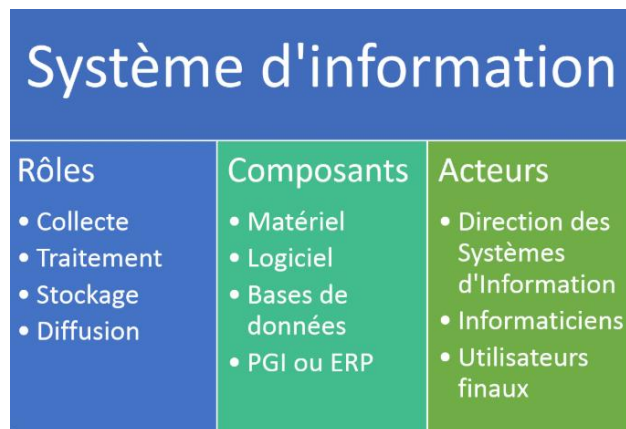
1.1 Systèmes d'Information (SI) dans l'Entreprise

A. Définition

Un Système d'Information (SI) est un ensemble *organisé* de ressources (matérielles, logicielles, humaines, financières, données, et procédures) qui permet de collecter, stocker, traiter et distribuer l'information nécessaire au fonctionnement et au pilotage de l'organisation.

Il ne se limite pas à l'informatique mais le *Système Informatique* qui est l'ensemble des moyens techniques automatisés : ordinateurs, logiciels, réseaux, bases de données, etc., est une composante du SI.

Objectif principal : Transformer les données brutes en information pertinente pour la prise de décision.



B. Architecture

L'architecture du SI est la structure qui décrit l'organisation et l'interaction des différentes composantes. Elle est souvent décomposée en trois niveaux :

1. **Couche Applicative** : Les applications/logiciels utilisés (ERP, CRM, applications métiers spécifiques, etc.) et comment ils échangent des données.
2. **Couche de Données** : La manière dont les données sont structurées, stockées, gérées (souvent via des SGBD) et sécurisées.
3. **Couche Technique** : L'infrastructure matérielle et réseau (serveurs, cloud, postes de travail, réseaux, etc.).

C. Rôle

Le SI a un rôle fondamental à plusieurs niveaux :

1. **Opérationnel** : Support aux processus quotidiens (gestion des commandes, production, paie, etc.).
2. **Tactique (ou de Gestion)** : Fournir des informations pour le pilotage et le contrôle (tableaux de bord, rapports d'activité).
3. **Stratégique** : Contribuer à l'atteinte des objectifs de l'entreprise, créer un avantage concurrentiel (par exemple, grâce à l'analyse de données clients, à l'innovation, etc.).

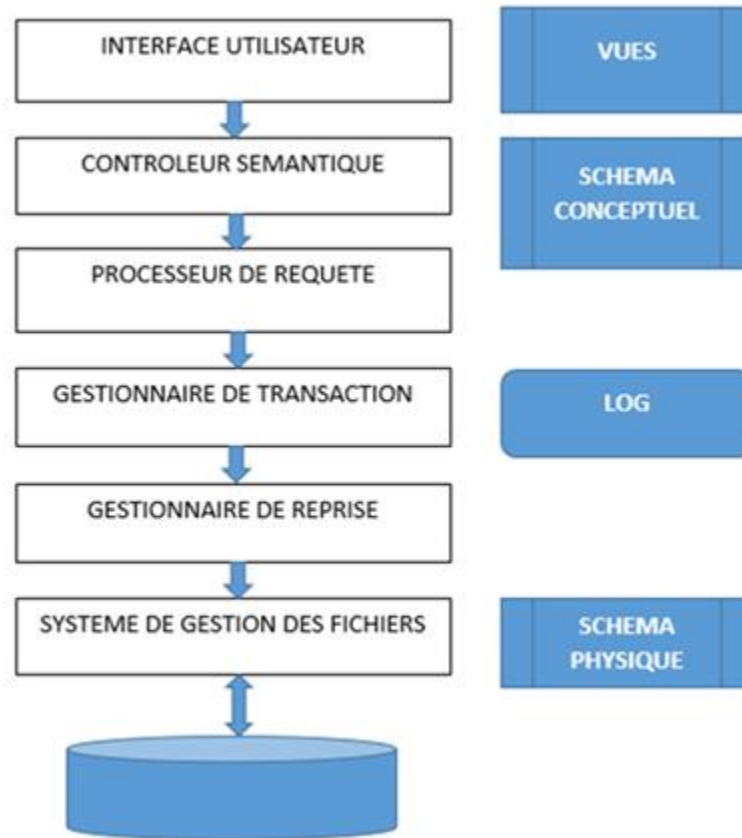
1.2. Évolution des Systèmes : Fichiers vs. SGBD

L'évolution du stockage et de la gestion des données a marqué une étape clé du SI.

Caractéristique	Systèmes basés sur Fichiers	Systèmes de Gestion de Bases de Données (SGBD)
Stockage	Données dispersées dans des fichiers indépendants.	Données centralisées et structurées dans une Base de Données (BD) .
Redondance	Élevée : la même information est souvent répétée dans plusieurs fichiers.	Minimale et contrôlée : l'information est stockée une seule fois.
Cohérence/Intégrité	Faible : difficile de garantir que toutes les copies d'une donnée sont à jour (mises à jour complexes).	Forte : le SGBD gère l'intégrité (règles, contraintes) et la cohérence des données.
Partage	Difficile et peu sécurisé.	Facilitée, simultanée par plusieurs utilisateurs, avec gestion des droits d'accès et de la concurrence.
Indépendance	Faible : la structure des données est souvent imbriquée dans le code des applications.	Forte (physique et logique) : les applications accèdent aux données par le SGBD, ce qui simplifie les modifications.

Le **SGBD** est devenu l'élément central du SI, offrant sécurité, intégrité, partage, et flexibilité que le simple stockage par fichiers ne pouvait garantir.

Architecture d'un SGBD



Chapitre 2 : Les données

La Donnée comme Actif Stratégique

Aujourd'hui, dans l'économie numérique, la **donnée** n'est plus un simple sous-produit des activités de l'entreprise, mais une **ressource stratégique** essentielle.

Définition

Les données sont considérées comme un actif car elles ont une **valeur économique** et peuvent être exploitées pour générer des revenus, améliorer l'efficacité opérationnelle et guider la stratégie.

Chapitre 3 : Fondements de la Modélisation Relationnelle

3.1 Concepts de Base de la Modélisation Relationnelle

La **modélisation relationnelle**, introduite par E.F. Codd en 1970, est le fondement de la plupart des bases de données modernes (**SGBDR** - Systèmes de Gestion de Bases de Données Relationnelles). Elle propose de représenter le **monde réel** en utilisant une **structure mathématique** simple basée sur la **théorie des ensembles** : la **relation**.

Cette approche s'articule autour de trois concepts fondamentaux : la **Relation**, l'**Attribut** et le **Tuple**.

3.1.1. La Relation (Le Tableau/La Table)

- **Définition Informelle** : La relation est l'équivalent d'un **tableau (table)**. Elle stocke un **ensemble d'informations de même nature**.
- **Définition Formelle** : Une relation est un **sous-ensemble du produit cartésien** des domaines des attributs qui la composent. Elle possède un **nom unique** dans la base de données.
- **Rôle** : Elle représente une **entité** (comme un client, un produit, une commande) ou une **association** entre plusieurs entités.
- **Exemples** : CLIENTS, COMMANDES, PRODUITS.

3.1.2. L'Attribut (Le Champ/La Colonne)

- **Définition Informelle** : L'attribut est un **champ** ou une **colonne** dans la table. Il représente **une propriété spécifique de la relation**.
- **Définition Formelle** : Chaque attribut est associé à un **domaine**, qui est l'ensemble des valeurs autorisées pour cet attribut (par exemple, le domaine d'un âge est l'ensemble des entiers positifs).
- **Rôle** : Il permet de **décrire l'entité** que la relation représente.
- **Exemples** : Pour la relation CLIENTS, les attributs pourraient être ID_Client, Nom, Adresse_Email. Pour la relation COMMANDES, ce pourrait être Date_Commande, Montant.

3.1.3 Le Tuple (L'Enregistrement/La Ligne)

- **Définition Informelle** : Le tuple est une **ligne** dans la table. Il représente une **seule occurrence** concrète de l'entité.
- **Définition Formelle** : Un tuple est une **liste ordonnée de valeurs**, où chaque valeur appartient au domaine de l'attribut correspondant.
- **Rôle** : Il contient les données d'un élément spécifique. L'ensemble des tuples constitue **l'extension de la relation** (le contenu actuel).
- **Exemples** : Un tuple dans la table CLIENTS contient toutes les données (ID, Nom, Email, etc.) pour **un** client spécifique.

3.1.4 Synthèse et Visualisation

Le concept de la modélisation relationnelle est souvent résumé par cette analogie :

Concept Relationnel	Analogie de la Table	Définition dans le Monde Réel
Relation	Le Tableau entier	Une Entité (ex: CLIENT)
Attribut	La Colonne/Le Champ	Une Propriété (ex: Nom)
Tuple	La Ligne	Une Occurrence (ex: Un client spécifique)

Exemple Concret : La Relation LIVRES

Attribut (Colonne) →	ISBN (Clé Primaire)	Titre	Auteur	Année_Publication (Domaine: Entier)
Tuple (Ligne)	978-207036034	1984	George Orwell	1949
Tuple (Ligne)	978-074327356	Gatsby le Magnifique	F. Scott Fitzgerald	1925
Tuple (Ligne)	978-150374189	Le Hobbit	J.R.R. Tolkien	1937

Dans cet exemple :

- Le tableau entier est la **Relation** LIVRES.
- ISBN, Titre, Auteur, Année_Publication sont les **Attributs**.
- Chaque ligne est un **Tuple**, représentant un livre unique (une occurrence de l'entité LIVRE).

3.2 L'Intégrité des Données

Pour assurer la fiabilité et la qualité des données , pour que ce modèle soit fonctionnel, il doit garantir l'unicité et la cohérence des données. C'est là qu'interviennent les concepts de **clés**.

3.2.1 Les Contraintes d'Intégrité Clés

3.2.1.1 La Clé Primaire (Primary Key - PK) : L'Identifiant Unique

La Clé Primaire est le mécanisme central pour garantir l'Intégrité d'Entité.

- **Définition** : C'est un attribut (colonne) ou un ensemble minimal d'attributs dont la valeur permet d'identifier de manière unique et sans ambiguïté chaque tuple (ligne) dans une relation (table).
- **Règles Fondamentales (Contrainte d'Intégrité d'Entité)** :
 1. **Unicité** : Deux tuples différents dans la même table ne peuvent pas avoir la même valeur pour la PK.
 2. **Non-Nullité** : La valeur de la Clé Primaire ne peut jamais être nulle (NULL). Chaque entité doit être identifiable.
- **Exemple** : Dans la table CLIENTS, l'attribut ID_Client est une Clé Primaire. Elle permet de garantir que chaque enregistrement client est distinct et qu'il n'y a pas d'ambiguïté pour le désigner.

3.2.1.2 La Clé Étrangère (Foreign Key - FK) : Le Lien entre les Tables

La Clé Étrangère est l'outil qui permet d'établir les relations entre les entités et de garantir l'Intégrité Référentielle.

- **Définition** : C'est un attribut ou un ensemble d'attributs dans une table (appelée table enfant ou table référençante) dont la valeur fait référence à la Clé Primaire (PK) d'une autre table (appelée table parent ou table référencée).
- **Règles Fondamentales (Contrainte d'Intégrité Référentielle)** :
 - Toute valeur présente dans la Clé Étrangère (FK) de la table enfant doit obligatoirement exister comme valeur dans la Clé Primaire (PK) de la table parent, ou elle doit être nulle (si la conception de la base le permet).
 - Ceci empêche la création de liens "brisés" ou de "fantômes" : on ne peut pas insérer une commande pour un client qui n'existe pas.
- **Exemple** :
 - Table Parent : CLIENTS (avec ID_Client comme PK).

- Table Enfant : COMMANDES (avec Num_Commande comme PK et ID_Client comme FK).
- Le champ ID_Client dans la table COMMANDES doit contenir une valeur qui existe réellement dans la colonne ID_Client de la table CLIENTS.

Visualisation de la Relation

Les clés fonctionnent ensemble pour construire la structure logique de la base de données.

Table CLIENTS (Parent)	
ID_Client (PK)	Nom
C001	Dupont
C002	Martin

Table COMMANDES (Enfant)	
Num_Commande (PK)	ID_Client (FK)
1001	C002
1002	C001
1003	C002

Conclusion sur l'Intégrité

L'utilisation correcte des Clés Primaires et Étrangères est la garantie que :

1. L'identité de chaque entité est assurée (grâce à la PK).
2. Les liens entre les entités sont maintenus et valides (grâce à la FK).

3.3 La normalisation

La normalisation est le processus qui consiste à organiser les colonnes et les tables d'une base de données pour :

- Minimiser la redondance des données.
- Améliorer l'intégrité et la cohérence.

La normalisation est un processus graduel où chaque forme normale impose des règles plus strictes que la précédente.

La **dépendance fonctionnelle** est le concept fondamental de la normalisation des bases de données relationnelles. Elle décrit une relation de contrainte entre deux ensembles d'attributs (colonnes) au sein d'une table.

En termes simples, il y a dépendance fonctionnelle lorsque la valeur d'un ensemble d'attributs (ou un seul attribut) détermine de manière unique la valeur d'un autre ensemble d'attributs.

La Règle de Détermination

Si un ensemble d'attributs A détermine fonctionnellement un ensemble d'attributs B , on l'écrit :

$$A \rightarrow B$$

- **A est le Déterminant** : C'est l'attribut (ou l'ensemble d'attributs) qui sert de "clé" ou de référence.
- **B est l'Attribut Dépendant** : C'est l'attribut dont la valeur est fixée par la valeur de A .

Analogie : Si vous connaissez A , alors vous connaissez nécessairement B . Pour chaque valeur de A , il n'existe qu'une seule valeur correspondante pour B .

Exemple Simple

Considérez une table `EMPLOYÉS` avec les colonnes `ID_Employé` et `Nom_Employé`.

$$\text{ID_Employé} \rightarrow \text{Nom_Employé}$$

- **Explication** : Chaque identifiant d'employé (`ID_Employé`) est unique et ne peut être associé qu'à un seul nom d'employé (`Nom_Employé`). Si vous connaissez l'ID, vous connaissez le Nom.
- **Inverse (Fausse)** : `Nom_Employé` $\nrightarrow$ `ID_Employé`
 - *Explication* : Deux employés peuvent avoir le même nom (ex: "Dupont"), donc connaître le nom n'est pas suffisant pour déterminer l'ID de manière unique.

3.3.1. Première Forme Normale (1FN) : Atomicité des Données

La 1FN est la forme la plus élémentaire et la base de toutes les autres.

Règle : Chaque colonne d'une table doit contenir uniquement des **valeurs atomiques** (indivisibles), et il ne doit y avoir **aucune colonne répétée** (groupe répétitif) au sein d'une seule ligne.

Atomique : Une cellule ne peut contenir qu'une seule information. Par exemple, une colonne Telephone ne doit pas contenir "0123456789, 0987654321". Il faudrait séparer cela en Telephone_1 et Telephone_2, ou, mieux, créer une table séparée TELEPHONES.

Pas de groupes répétitifs : On ne doit pas avoir des colonnes comme Produit_1, Produit_2, Produit_3 dans une table COMMANDES. Chaque produit doit avoir sa propre ligne dans une table de détail.

3.3.2. Deuxième Forme Normale (2FN) : Dépendance Complète vis-à-vis de la Clé

Pour qu'une table soit en 2FN, elle doit d'abord être en 1FN.

Règle : Toute colonne qui ne fait pas partie de la clé primaire doit dépendre **entièrement** de la **clé primaire complète**.

Concerne : Principalement les tables ayant une **clé primaire composite** (composée de deux colonnes ou plus).

Dépendance Partielle (Violation de la 2FN)

Une **dépendance partielle** se produit lorsque la clé primaire de la table est **composite** (composée de plusieurs attributs), et qu'un attribut non-clé ne dépend que d'une **partie** de cette clé.

Si une colonne dépend uniquement d'une *partie* de la clé primaire composite, elle doit être déplacée dans une nouvelle table où cette partie de la clé devient sa clé primaire.

Table (Clé : A, B)	Dépendance	Violation	Correction (2FN)
(<u>A</u> , <u>B</u> , C, D)	$A \rightarrow C$	C dépend seulement de A (une partie de la clé)	Déplacer A et C dans une nouvelle table (<u>A</u> , C).

Exemple : Dans une table LIGNES_COMMANDE (ID_Commande, ID_Produit, Quantité, Prix_Produit), si le Prix_Produit dépend uniquement de ID_Produit et non de l'ensemble (ID_Commande, ID_Produit), il y a une dépendance partielle. Le Prix_Produit doit être déplacé dans une table PRODUITS.

3. Troisième Forme Normale (3FN) : Pas de Dépendance Transitive

Pour qu'une table soit en 3FN, elle doit d'abord être en 2FN.

Règle : Aucune colonne qui ne fait pas partie de la clé primaire ne doit dépendre d'une autre colonne qui ne fait pas partie de la clé primaire. C'est l'élimination des **dépendances transitives**.

Explication : Si $A \rightarrow B$ et $B \rightarrow C$, alors on a une dépendance transitive $A \rightarrow C$. La 3FN exige de supprimer la dépendance $B \rightarrow C$ en déplaçant B et C dans une nouvelle table.

Table (Clé : A)	Dépendance	Violation	Correction (3FN)
$(\underline{A}, B, C)$	$A \rightarrow B$ et $B \rightarrow C$	C dépend de B , qui n'est pas la clé primaire.	Déplacer B et C dans une nouvelle table $(\underline{B}, C)$.

Exemple : Dans une table CLIENTS (ID_Client, Nom_Client, ID_Ville, Nom_Ville), Nom_Ville dépend de ID_Ville, et ID_Ville dépend de ID_Client. Nom_Ville ne dépend donc pas directement de la clé primaire (ID_Client). La solution est de créer une table VILLES (ID_Ville, Nom_Ville) et de laisser seulement ID_Ville dans la table CLIENTS.

En général, atteindre la **3FN** est considéré comme suffisant pour la plupart des applications de bases de données, car cela **minimise la redondance** et assure une **bonne intégrité** sans complexifier excessivement le schéma.

En identifiant et en corrigeant ces dépendances problématiques via la décomposition des tables, on s'assure que chaque information est stockée une seule fois, atteignant ainsi les objectifs de cohérence et de minimalisation de la redondance.

CHAPITRE 4 Modèle Conceptuel de Données (MCD)

Le MCD décrit **ce que** le système doit mémoriser, en utilisant un langage proche de l'utilisateur. Il est **indépendant** de tout choix technique (logiciel de base de données, langage de programmation).

- **Composants principaux (selon la notation Entité-Association, souvent Merise) :**
 - **Entité (ou Type d'Entité) :** Représente un ensemble d'objets de même nature (ex: Client, Produit).
 - **Propriété (ou Attribut) :** Une information décrivant l'entité (ex: Nom, Adresse pour Client).
 - **Identifiant :** La propriété qui permet de distinguer de manière unique une occurrence dans l'entité (ex: NuméroClient).
 - **Association (ou Type de Relation) :** Un lien entre plusieurs entités (ex: "Commander" lie Client et Produit).

- **Cardinalités** : Elles définissent les règles de gestion (minimum et maximum) pour la participation d'une entité à une association (ex: (1, N) signifie "au moins 1, au plus plusieurs").

Exemples MCD :

Considérons un système de gestion de commandes simple.

- **Entités** : CLIENT, COMMANDE
- **Association** : PASSER entre CLIENT et COMMANDE.
- **Règles** :
 - Un CLIENT **passse** au moins 0 et au plus plusieurs COMMANDE (0, N).
 - Une COMMANDE est **passée** par un et un seul CLIENT (1, 1).

1. Exemple : Gestion de Bibliothèque

- **Entités** :
 - **Adhérent** (ID_Adhérent, Nom, Prénom, Adresse).
 - **Livre** (ISBN, Titre, Auteur).
- **Association** :
 - **Emprunter** : Un Adhérent peut emprunter 0 à N Livres. Un Livre est emprunté par 0 à 1 Adhérent.
- **Cardinalités** :
 - Adhérent (0,N) ↔ Emprunter ↔ (0,1) Livre. 

2. Exemple : E-commerce (Vente)

- **Entités** :
 - **Client** (ID_Client, Nom, Email).
 - **Produit** (ID_Produit, Désignation, Prix).
 - **Commande** (Num_Commande, Date).
- **Associations** :
 - **Passer** : Client (1,N) ↔ Commande.
 - **Contenir** : Commande (1,N) ↔ Produit (0,N). 

3. Exemple : Système de Blog (Utilisateur/Article)

- **Entités** : Utilisateur, Article, Catégorie, Avatar.
- **Exemples de règles de gestion** :
 - Un **Utilisateur** publie 0 à N **Articles**.
 - Un **Article** est publié par 1 et 1 seul **Utilisateur**.
 - Un **Article** possède 0 à N **Catégories**. 

2. Modèle Logique de Données (MLD)

Le MLD est la traduction du MCD dans un modèle spécifique, généralement le **modèle relationnel** (le plus utilisé). Il décrit **comment** les données seront structurées sous forme de **tables**.

- **Règles de passage MCD vers MLD** :
 1. **Chaque entité** du MCD devient une **table** dans le MLD. L'identifiant devient la **clé primaire (Primary Key - PK)**.
 2. **Les associations** sont gérées en ajoutant des colonnes appelées **clés étrangères (Foreign Key - FK)** dans les tables.

Application de l'Exemple au MLD :

1. **Entité CLIENT → Table CLIENT**
 - CLIENT (*NumClient*, NomClient, Ville...) → PK = NumClient
2. **Entité COMMANDE → Table COMMANDE**
 - COMMANDE (*NumCommande*, DateCommande...) → PK = NumCommande
3. **Association PASSER (Type 1, 1 - 0, N) →** La clé primaire de l'entité côté (1, 1) (CLIENT) migre vers la table côté (0, N) (COMMANDE) comme clé étrangère.
 - COMMANDE (*NumCommande*, DateCommande, **#NumClient**) → FK = NumClient

MLD Final (Relationnel) :

- **CLIENT** (**NumClient** , NomClient, Ville)
- **COMMANDE** (**NumCommande** , DateCommande, **#NumClient**)

Note : Le symbole # est souvent utilisé pour désigner une Clé Étrangère.

3. Modèle Physique de Données (MPD)

Qu'est-ce que c'est ?

Le MPD est la traduction concrète du MLD dans un **Système de Gestion de Base de Données (SGBD)** spécifique (ex: MySQL, PostgreSQL, Oracle). Il décrit **où** et **comment** les données sont stockées *physiquement*.

Composants additionnels par rapport au MLD :

- **Types de données spécifiques** (ex: VARCHAR(50), INT, DATE).
- **Contraintes d'intégrité** (ex: NOT NULL, UNIQUE).
- **Index** pour optimiser les performances de recherche.
- **Nommage** conforme aux conventions du SGBD.

Application de l'Exemple au MPD :

Nous allons choisir **PostgreSQL** comme SGBD.

- **Table CLIENT :**

- NumClient (PK) devient INTEGER PRIMARY KEY
- NomClient devient VARCHAR(100) NOT NULL
- Ville devient VARCHAR(50)

- **Table COMMANDE :**

- NumCommande (PK) devient INTEGER PRIMARY KEY
- DateCommande devient DATE NOT NULL
- #NumClient (FK) devient INTEGER REFERENCES CLIENT(NumClient) NOT NULL

4. Les Tables (Implémentation concrète)

Qu'est-ce que c'est ?

C'est la phase d'implémentation, où le MPD est traduit en commandes **SQL (Structured Query Language)** pour créer les structures de la base de données (le **schéma**).

Application de l'Exemple aux Tables (Code SQL) :

Voici le code SQL (langage DDL - Data Definition Language) qui crée concrètement les tables dans le SGBD :

SQL

-- Création de la TABLE CLIENT

CREATE TABLE CLIENT (

NumClient INTEGER PRIMARY KEY, -- Clé Primaire

```

    NomClient VARCHAR(100) NOT NULL,

    Ville VARCHAR(50)

);

-- Création de la TABLE COMMANDE

CREATE TABLE COMMANDE (

    NumCommande INTEGER PRIMARY KEY,    -- Clé Primaire

    DateCommande DATE NOT NULL,

    NumClient INTEGER NOT NULL,        -- Clé Étrangère (FK)

    -- Définition de la contrainte de Clé Étrangère

    FOREIGN KEY (NumClient) REFERENCES CLIENT(NumClient)

);

```

Représentation des données dans les Tables :

Une fois les tables créées, on peut y insérer des données :

CLIENT	
NumClient (PK)	NomClient
1	Dupont
2	Martin

COMMANDE	
NumCommande (PK)	DateCommande
101	2025-01-15
102	2025-01-16

COMMANDE	
103	2025-01-16

Synthèse

Étape	Rôle / Question	Indépendance	Composants Clés	Exemple
MCD	Quoi ? (Conception des faits)	Totalement indépendant de la technique	Entités, Associations, Cardinalités	CLIENT (1,N) PASSER (1,1) COMMANDE
MLD	Comment ? (Organisation logique)	Indépendant du SGBD	Tables, Clés Primaires (PK), Clés Étrangères (FK)	COMMANDE (#NumClient)
MPD	Où ? (Mise en œuvre technique)	Dépendant du SGBD (Oracle, MySQL...)	Types de données, Index, Contraintes (NOT NULL)	NumClient INTEGER PRIMARY KEY
Tables	Implémentation (Code SQL)	Dépendant du SGBD	Code CREATE TABLE	CREATE TABLE CLIENT (...)